

Computer Vision – Individual Lab 2

Pham Minh Hoang

March 22, 2026

Problem

In this assignment, you have to implement from scratch the Harris key point detection algorithm using OpenCV C/C++. Your program is called by command line arguments and perform the following functions

1. Load the input image from its path
2. Show the result image and save it. The keypoints are highlighted by drawing circles. You can use *cv::circle* to draw circles in OpenCV
3. Detect Harris keypoints. The command line is

```
<Executable file> -harris <InputFilePath> <OutputFilePath>
```

where

- *-harris*: the name of command
 - *InputFilePath*: the path of input file
 - *OutputFilePath*: the path of output file
4. Optimizing the Harris Keypoint Detection algorithm using pixel manipulation via image pointers and computational complexity reduction methods. The command line is

```
<Executable file> -optHarris <InputFilePath> <OutputFilePath>
```

where

- *-optHarris*: the name of command
- *InputFilePath*: the path of input file
- *OutputFilePath*: the path of output file

Requirements

1. Library Restrictions

- **Prohibited Functions:** Students are strictly prohibited from using any built-in OpenCV image processing functions (e.g., `cv::cornerHarris`, `cv::Sobel`, `cv::GaussianBlur`, `cv::Canny`, etc.).
- **Permitted Functions:** Only basic I/O and visualization functions are allowed: `cv::imread`, `cv::imwrite`, `cv::imshow`, and `cv::circle` (solely for marking detected points).
- **Algorithm Implementation:** All core logic, including gradient calculation, must be implemented **from scratch**.

2. Pixel Processing

- **Pointer-based Processing:** Manual pixel manipulation via **raw pointers** (`uchar*` or `float*`) is mandatory. Use of `cv::Mat::at<T>(y, x)` or `cv::MatIterator` is not allowed.
- **Pointer Arithmetic:** Students are encouraged to use efficient pointer arithmetic (e.g., `*(ptr + y * step + x)`) to demonstrate an understanding of image memory layout (stride/step).

3. Code Quality

- **Line-by-line Comments:** Students must provide detailed comments explaining the logic and purpose of each line of processing code.
- **Function Comments:** For every function implemented, students must clearly comment: the function's purpose, the meaning of each input parameter, and the meaning of the the return parameter.

Submission

Your submission must organize into 4 folders and compressed in 1 file *StudentID.zip*

1. Document (contains your own report, personal information, user guide, your proposed optimization algorithm)
2. Data (contains input images and output images)
3. Executable (contain executable file). The executable file is named *StudentID*
4. Sources (contain all source code built successfully).